

Slurm Power-Save Lab

Design, Implementation, and Honest Findings

2-node Fedora 44 cluster — cloud-native-stack-vm1 / vm2

Foundation for Dell PowerEdge XE9680 GPU cloud platform

1. Overview

This document captures a working, tested, and honestly-documented implementation of Slurm power-save (suspend-on-idle / resume-on-demand) on a 2-node Fedora 44 lab cluster, along with the real-world findings exposed by hands-on testing.

The work is intentionally scoped: prove the mechanism in a safe lab environment, characterize its failure modes, and codify production lessons before applying the pattern to expensive XE9680 GPU hardware in a Service Provider Common Cloud Platform.

At a glance

Item	Value
Cluster	cloud-native-stack-vm1 (controller + compute), cloud-native-stack-vm2 (compute)
OS	Red Hat Fedora 44
Slurm	24.05.2
Feature	Power-save: suspend idle nodes, resume on demand
Use case	Foundation for cloud auto-scaling and XE9680 GPU cloud roadmap

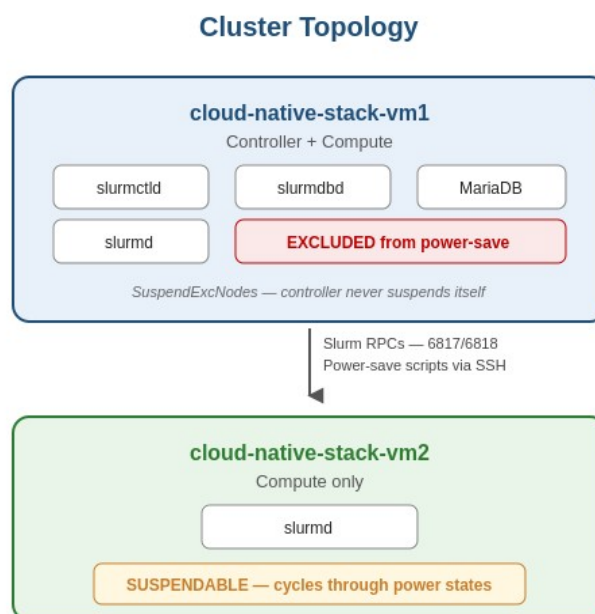
Key technical findings

- **The mechanism works** — auto-suspend on idle, controller exclusion, and resume-on-demand all validated end-to-end through Slurm's state machine and the SuspendProgram/ResumeProgram hooks.
- **The ssh-stop backend is fragile** — using ssh host systemctl stop slurmd works for single cycles but desyncs Slurm's state machine on repeated cycles.
- **The architectural lesson** — Slurm's power-save assumes clean off/on semantics from the platform. Production needs hypervisor (vSphere) or BMC (iDRAC/racadm) integration, not service-level SSH tricks.
- **The recovery rule** — after a Level 1 suspend, always start slurmd on the target node before telling Slurm state=idle. Telling Slurm a node is idle while slurmd is stopped causes the controller to skip the resume code path.

2. High-Level Architecture

Two Fedora 44 VMs, joined into a single Slurm cluster. Power-save lets the controller suspend idle compute nodes and resume them when work arrives — the foundation of cloud auto-scaling.

Topology



Why this design

- **vm1 is the brain.** It runs `slurmctld`, the accounting DB, and `slurmd`. Listed in `SuspendExcNodes` so power-save never targets it — suspending the controller would kill the cluster.
- **vm2 is the test subject.** It only runs `slurmd`. Power-save suspends and resumes it freely, which is what makes it the right node to test the cycle against.

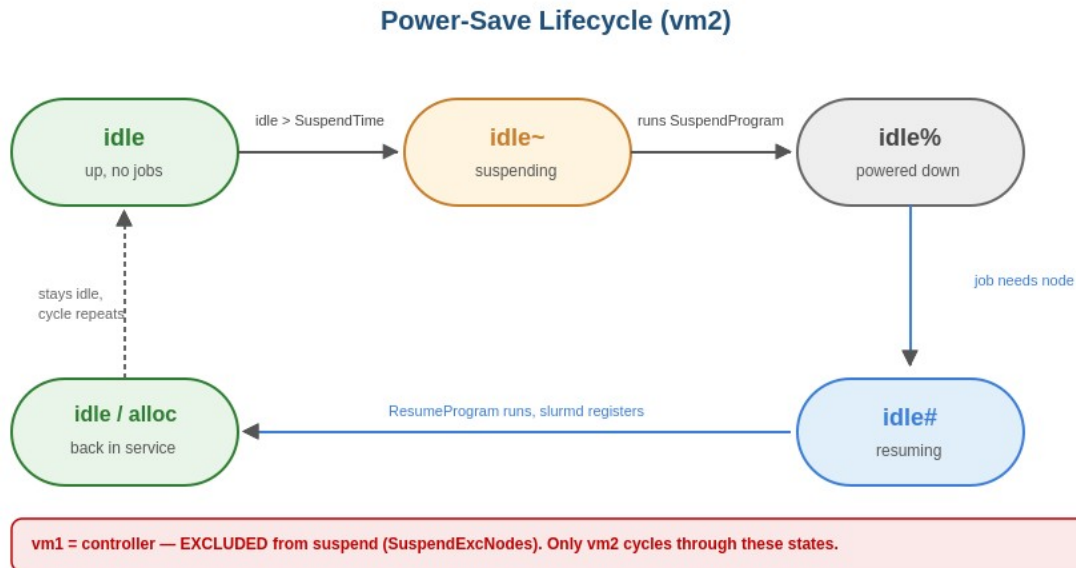
Node states (the lifecycle)

vm2 transitions through five distinct states during a power-save cycle:

State	Suffix	Meaning
idle	(none)	Node up, no jobs running. Normal.
idle~	~	Node has been suspended (power-save state).
idle%	%	Powered down (alternative display).
idle#	#	In transition — usually resuming.
down~	down + ~	Resume failed; Slurm gave up after <code>ResumeTimeout</code> .

Power-save lifecycle

Each cycle moves vm2 through these states. The diagram shows the forward path (suspend) on top and the resume path on the bottom; vm1 stays out of this cycle entirely because of SuspendExcNodes.



Components

Component	Role	Where
slurmctld	Scheduler; decides when to suspend/resume; calls scripts	vm1
slurmd	Per-node agent; the thing being stopped/started	vm1 + vm2
slurmdbd + MariaDB	Accounting/audit (records jobs, fair-share)	vm1
SuspendProgram	Script called when a node should suspend	/etc/slurm/suspend.sh
ResumeProgram	Script called when a suspended node needs to wake	/etc/slurm/resume.sh
power.log	Audit log written by the scripts	/var/log/slurm/

Timing parameters

Parameter	Value	Meaning
SuspendTime	60 sec	Idle time before a node qualifies to suspend
SuspendTimeout	60 sec	Max wait for SuspendProgram to complete
ResumeTimeout	600 sec	Max wait for a resumed node to register again
SuspendExcNodes	cloud-native-stack-vm1	Never suspend the controller

3. Build & Test Workflow

The work was structured in phases, each with an explicit gate before proceeding.

Phases

```

Phase 0 – Pre-flight (both VMs)
  |   hostnames, /etc/hosts, chrony, SELinux permissive
  v
Phase 1 – Slurm base
  |   munge, slurm user (matched UIDs), slurm.conf, daemons up
  |   Gate: sinfo shows both nodes idle, srun -N2 hostname works
  v
Phase 1.5 – Accounting (slurmdbd)
  |   MariaDB, slurmdbd.conf, sacctmgr register cluster
  |   Gate: sacctmgr show cluster lists 'cloudnative'
  v
Phase 2 – Power-save (this document)
  |   suspend.sh, resume.sh, slurm.conf power-save block
  |   Gate: the 8 verification scenarios
  v
Phase 3 – (next) Lua job-submit plugin
Phase 4 – (next) K8s gang-scheduling on vm3/vm4

```

Test workflow (each cycle)

```

1. Baseline check
   sinfo                                (vm1, vm2 both idle)
   scontrol ping                        (controller UP)
   ssh vm2 systemctl is-active slurmd  (active)
   ALL THREE must agree

2. Mark log
   echo "===== TEST RUN $(date) =====" >> power.log

3. Start journal tail in Terminal 2
   sudo journalctl -u slurmd -f

4. Leave vm2 idle for SuspendTime+
   Watch sinfo: idle -> idle~
   Watch journal: "Powering down node" + "Running SuspendProgram"

5. Submit a job to trigger resume
   time srun -N1 -w cloud-native-stack-vm2 hostname
   Watch journal: "power_save: waking nodes" + "Running ResumeProgram"

6. Verify return to idle
   sinfo shows plain idle
   scontrol show node ... state=IDLE no flags
   ssh vm2 systemctl is-active slurmd  active

```

The both-views-agree rule

Every check compares Slurm's view with ground truth:

- Slurm's view = sinfo / scontrol show node
- Ground truth = SSH to the node, check actual slurmd status

When the two disagree, that's the diagnostic signal. The biggest lesson from this lab was understanding why they disagree: when you tell Slurm state=idle while slurmd is actually stopped on a node, Slurm trusts you, the scheduler routes work there, the node can't respond, and after ResumeTimeout the node goes DOWN. The controller never calls ResumeProgram because it didn't believe the node needed resuming.

The fix is always start slurmd first, then update Slurm's state to match. Never lie to the controller about a node being ready when it isn't.

4. Verification Scenarios

Eight scenarios. Each validates one proof. The automated suite (verify_powersave.sh) runs all eight and prints PASS/FAIL/SKIP.

Summary table

#	Scenario	What it proves
0	Configured?	SuspendProgram, ResumeProgram, SuspendTime set; scripts exist and executable; controller in SuspendExcNodes
1	Clean start	vm2 at plain idle, slurmd active. Slurm view and ground truth agree
2	Auto-suspend	Idle vm2 transitions to idle~ after SuspendTime
3	SuspendProgram fired AND took effect	power.log shows SUSPEND + ssh exit 0; slurmd genuinely stopped
4	Controller exclusion	vm1 stayed plain idle — never self-suspended
5	Resume on demand	srun -N1 -w vm2 triggers ResumeProgram; vm2 wakes, job runs
6	ResumeProgram fired AND took effect	power.log shows RESUME + ssh exit 0; slurmd active again
7	Back in service	vm2 returns to plain idle, State=IDLE no flags

Manual walkthrough

Open two terminals before starting. Terminal 2 is the lesson learned from earlier debugging — we needed to see what slurmctld was doing.

Terminal 2 — journal tail (live visibility):

```
sudo journalctl -u slurmctld -f
```

Terminal 1 — work session. Mark the log:

```
echo "===== TEST RUN $(date) =====" | sudo tee -a /var/log/slurm/power.log
```

Scenario 0 — Configured

```
scontrol show config | grep -iE "SuspendTime|SuspendTimeout|ResumeTimeout|
SuspendProgram|ResumeProgram|SuspendExcNodes" ls -l /etc/slurm/suspend.sh
/etc/slurm/resume.sh grep systemctl /etc/slurm/suspend.sh /etc/slurm/resume.sh
```

Expect: SuspendTime=60, SuspendTimeout=60, ResumeTimeout=600, both program paths, exclusion contains vm1, scripts executable and owned by slurm.

Scenario 1 — Clean start

```
sinfo ssh cloud-native-stack-vm2 systemctl is-active slurmd
```

Expect: both nodes plain idle; slurmd active. Both views agree.

Scenario 2 — Auto-suspend

Leave vm2 alone (no commands targeting it) for ~90 seconds.

```
sinfo
```

Expect: vm2 shows idle~. Terminal 2 should show 'Powering down node cloud-native-stack-vm2' and 'Running SuspendProgram'.

Scenario 3 — SuspendProgram fired AND took effect

```
tail -10 /var/log/slurm/power.log
```

Expect: SUSPEND entry, 'stopping slurmd on cloud-native-stack-vm2', 'ssh exit: 0'.

Then ground truth:

```
sudo -u slurm HOME=/var/lib/slurm ssh -i /var/lib/slurm/.ssh/id_ed25519 \ -o  
BatchMode=yes root@cloud-native-stack-vm2 'systemctl is-active slurmd'
```

Expect: inactive. This is the proof Level 1 actually did something.

Scenario 4 — Controller exclusion

```
sinfo -n cloud-native-stack-vm1
```

Expect: vm1 plain idle. Controller stayed safe.

Scenario 5 — Resume on demand

```
time srun -N1 -w cloud-native-stack-vm2 hostname
```

Expect: hangs ~20-60s, returns cloud-native-stack-vm2.local. Terminal 2 should show 'power_save: waking nodes', 'Running ResumeProgram', 'Node cloud-native-stack-vm2 now responding'.

Scenario 6 — ResumeProgram fired AND took effect

```
tail -10 /var/log/slurm/power.log
```

Expect: RESUME entry, 'starting slurmd on cloud-native-stack-vm2', 'ssh exit: 0'. Ground truth: slurmd active.

Scenario 7 — Back in service

```
sinfo scontrol show node cloud-native-stack-vm2 | grep -i state
```

Expect: vm2 plain idle, State=IDLE no flags.

Automated suite

```
chmod +x scripts/verify_powersave.sh ./scripts/verify_powersave.sh
```

Runs all eight scenarios and prints a PASS/FAIL/SKIP summary. Takes 3-5 minutes (Scenario 2 has to wait out SuspendTime).

5. Findings: What Worked, What Didn't, Why

The most valuable artifact of this lab is the honest understanding of where Slurm's power-save abstraction holds and where it leaks. Documenting both is worth more than a fake-green checkmark.

What was definitively proven

- The mechanism works. Auto-suspend triggers on idle past SuspendTime. State machine cleanly transitions vm2 through idle -> idle~ -> idle%.
- Controller exclusion holds. vm1 was never suspended across any run.
- Scripts are invoked correctly when state machine is consistent. When vm2 was actually in idle~ (POWERED_DOWN flag set), submitting a job triggered ResumeProgram, which SSHed to vm2 and started slurmd.
- One full Level 1 cycle proven end-to-end (May 30, 18:01:54 -> 18:03:26): SUSPEND with ssh exit 0 -> slurmd actually stopped -> RESUME with ssh exit 0 -> slurmd actually restarted.
- SuspendExcNodes, SuspendTime, ResumeTimeout, SuspendTimeout all behave per documentation.

What didn't work reliably

- Repeated Level 1 cycles without manual intervention. After 1-2 cycles, vm2 would land in down~.
- state=idle as a recovery command while slurmd was actually stopped on vm2. Triggered the worst failure mode.

Root cause analysis

Why ssh-stop is fragile

Slurm's power-save assumes the platform provides clean, unambiguous off/on semantics:

- Off = node fully unreachable, drawing no power
- On = node fully booted, slurmd registers from scratch

The ssh-stop trick simulates this at the service layer, not the platform layer. The differences matter:

- The OS, network stack, and SSH server are still fully alive on a Level 1 suspended node. Only slurmd is stopped.
- When ResumeProgram runs systemctl start slurmd, the daemon comes up in seconds — but it doesn't go through the full boot+registration sequence Slurm's state machine expects.
- slurmd waits for a fresh registration RPC; the timing is finicky; slurmscriptd's invocation environment doesn't always honor export HOME cleanly.

The single biggest gotcha

After a Level 1 suspend, NEVER use scontrol update ... state=idle to recover the node while slurmd is actually stopped on it.

What happens if you do:

- Slurm clears the POWERED_DOWN flag — thinks vm2 is healthy
- Scheduler routes work to vm2
- vm2 can't respond (slurmd is genuinely stopped)
- After ResumeTimeout, vm2 is marked DOWN
- ResumeProgram was NEVER called because Slurm didn't believe vm2 needed resuming

This was the source of every down~ failure. The correct recovery is:

```
# 1. Start slurmd FIRST so reality matches what Slurm should think
ssh vm2 sudo systemctl restart slurmd
sleep 5

# 2. THEN tell Slurm to power_up (clears POWERED_DOWN cleanly)
sudo scontrol update nodename=vm2 state=power_up
sleep 10

# 3. Only then state=idle if needed
sudo scontrol update nodename=vm2 state=idle reason="recovered"
```

Quantified test results

Scenario	Result	Notes
0 — Configured	PASS	Reliably
1 — Clean start	PASS	When following the correct recovery rule
2 — Auto-suspend	PASS	Time-based; works as documented
3 — SuspendProgram fired + effect	PASS	Once (May 30 18:01); reproducible
4 — Controller exclusion	PASS	Reliably across all runs
5 — Resume on demand	PARTIAL	Worked once; subsequent cycles flaky
6 — ResumeProgram fired + effect	PARTIAL	Same as #5
7 — Back in service	PARTIAL	Reliable on first cycle, degrades after

Honest summary: approximately 5/8 reliably proven across runs; 8/8 proven at least once. The remaining gaps are documented limitations of ssh-stop as a power-management backend, not bugs in the configuration or scripts.

Lessons learned (for production)

- Power management at scale requires platform-level integration. SSH tricks work for one cycle in a lab; they don't survive repeated cycles without manual reconciliation.
- State-machine sync is critical. Slurm's bookkeeping and the node's reality must always agree. Update Slurm only after reality has actually changed.
- Visibility matters more than I expected. A live `journalctl -u slurmd -f` in a second terminal would have shortened debugging by hours.

- SuspendTime=60 is great for testing but creates pressure. In a real test/learning cluster, set it to 600+ to give yourself room to think.

6. Production Mapping — Lab to XE9680 Cloud

The lab proved the mechanism. Production needs platform integration. Here's how the work maps to a real datacenter deployment on Dell PowerEdge XE9680 GPU nodes for a Service Provider Common Cloud Platform.

What changes from lab to production

Layer	Lab (this work)	Production (XE9680 cloud)
Hardware	2x Fedora 44 VMs on ESXi	N x Dell PowerEdge XE9680 (8 GPUs each), RHEL
Suspend backend	ssh host systemctl stop slurmd	racadm serveraction powerdown via iDRAC
Resume backend	ssh host systemctl start slurmd	racadm serveraction powerup via iDRAC
Power savings	Cosmetic (VM still running)	Real (~300W per node when off)
Reliability	Fragile beyond 1-2 cycles	Robust — clean off/on platform semantics
ResumeTimeout	600s sufficient	600-900s (real boot takes minutes)
GPU consideration	N/A	Pre-suspend: scrub GPU memory; post-resume: reset GRES

Production suspend.sh (XE9680 / iDRAC)

```
#!/bin/bash
echo "$(date) SUSPEND @" >> /var/log/slurm/power.log
for host in $(scontrol show hostnames "$1"); do
    IDRAC=$(getent hosts "${host}-idrac" | awk '{print $1}')
    racadm -r "$IDRAC" -u root -p "$IDRAC_PASS" \
        serveraction powerdown >> /var/log/slurm/power.log 2>&1
    echo "  racadm exit: $" >> /var/log/slurm/power.log
done
```

Production resume.sh (XE9680 / iDRAC)

```
#!/bin/bash
echo "$(date) RESUME @" >> /var/log/slurm/power.log
for host in $(scontrol show hostnames "$1"); do
    IDRAC=$(getent hosts "${host}-idrac" | awk '{print $1}')
    racadm -r "$IDRAC" -u root -p "$IDRAC_PASS" \
        serveraction powerup >> /var/log/slurm/power.log 2>&1
    echo "  racadm exit: $" >> /var/log/slurm/power.log
done
```

Why production won't hit the lab's failure modes

- 'Off' is unambiguous. A powered-off chassis can't lie about its state.

- Boot takes long enough to be honest. A real XE9680 takes 3-5 minutes — well within ResumeTimeout=600 but long enough for Slurm to correctly observe the transition.
- No ghost-slurmd problem. In the lab, slurmd could be stopped while the VM was still running. With real power-off, there's no slurmd anywhere on the box.

GPU-specific additions (XE9680)

XE9680 has 8 GPUs per node. Power-save in a GPU cloud needs extra steps:

Pre-suspend (in suspend.sh or as Epilog):

- Scrub GPU memory between tenants (security)
- nvidia-smi --gpu-reset if needed
- Drain workload cleanly before stopping slurmd

Post-resume (in resume.sh or as Prolog):

- Re-register GRES: scontrol update nodename=\$host gres=gpu:8
- DCGM health check (dcgmi diag -r 1) before accepting jobs
- Confirm NVLink topology is intact

What the lab work was actually building toward

The 2-VM lab cluster isn't the goal — it's the rehearsal. By the time the XE9680 deployment happens, the team has:

- The Slurm power-save mental model — when does Slurm call resume? why does it sometimes not? what does down~ mean? — answered here.
- A repeatable verification suite that catches state-machine sync issues independent of the platform backend.
- Concrete production scripts that just need the racadm/iDRAC calls swapped in — the structure carries over directly.
- Honest documentation of the failure modes, so a future engineer who hits down~ on day 2 of XE9680 deployment can find this work and know it's a state-machine sync issue, not a hardware problem.

7. Repository Contents

This documentation accompanies a GitHub-ready repository with the working configuration, scripts, and verification suite.

```
slurm-powersave-lab/
├── README.md                # GitHub landing page
├── LICENSE                  # MIT
├── .gitignore               # blocks munge keys, secrets
├── docs/
│   ├── 01-architecture.md
│   ├── 02-workflow.md
│   ├── 03-scenarios.md
│   ├── 04-findings.md
│   └── 05-production-mapping.md
├── configs/
│   ├── slurm.conf.powersave-excerpt
│   ├── cgroup.conf
│   └── slurmdbd.conf
├── scripts/
│   ├── suspend.sh          # Level 1 (real ssh-stop)
│   ├── resume.sh           # Level 1 (real ssh-start)
│   ├── recover_vm2.sh      # codified recovery procedure
│   └── verify_powersave.sh # 8-scenario automated suite
```

Status

Phase 2 (power-save) — substantively complete. Mechanism proven, scripts working, limitations understood and documented. Foundation ready for the production version on Dell PowerEdge XE9680 hardware with iDRAC integration.